



# **Pengembangan Sistem Manajemen Database Terintegrasi untuk Berkat Dinasti dalam Mendukung Digitalisasi UMKM di Indonesia**



## **PENCIPTA**

Dzauq Bachrul 'Ulum (250535621835)

Aditya Widyatamaka Tri Utomo (250535623587)

Chasbiah Azzahra (250535621731)

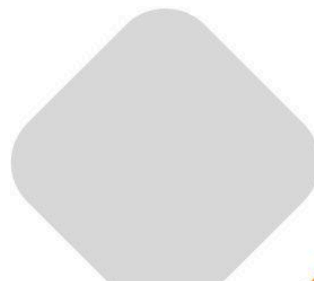
Elshifa Viorissa (250535621859)

Dr. Ir. Triyanna Widiyaningtyas, M.T.

Ir. Ilham Ari Elbaith Zaeni, S.T., M.T., Ph.D.

Harits Ar Rosyid, S.T., M.T., Ph.D.

Wahyu Muhammad Noble Shafa



## **I. Judul**

Pengembangan Sistem Manajemen Database Terintegrasi untuk Berkat Dinasti dalam Mendukung Digitalisasi UMKM di Indonesia

## **II. Latar Belakang**

Perkembangan teknologi informasi di era digital saat ini membawa perubahan besar dalam berbagai sektor, termasuk sektor Usaha Mikro, Kecil, dan Menengah (UMKM). Digitalisasi menjadi salah satu faktor penting dalam meningkatkan daya saing UMKM di Indonesia. Dengan memanfaatkan teknologi, UMKM dapat mengelola data, meningkatkan efisiensi operasional, serta memperluas jangkauan pasar.

Namun, pada kenyataannya masih banyak UMKM yang belum menerapkan sistem pengelolaan data yang terstruktur dan terintegrasi. Proses pencatatan yang masih dilakukan secara manual seringkali menimbulkan berbagai permasalahan, seperti kesalahan pencatatan, duplikasi data, serta kesulitan dalam melakukan analisis dan pengambilan keputusan.

Berkat Dinasti sebagai salah satu pelaku UMKM di bidang kuliner juga menghadapi permasalahan serupa. Pengelolaan data transaksi dan stok barang yang belum terintegrasi menyebabkan kurang optimalnya kinerja operasional serta pengambilan keputusan bisnis.

Melalui tugas akhir mata kuliah Basis Data pada Program Studi Teknik Informatika Universitas Negeri Malang, mahasiswa diberikan kesempatan untuk mengimplementasikan konsep-konsep yang telah dipelajari ke dalam dunia nyata melalui kerja sama dengan mitra usaha. Dalam hal ini, tim mahasiswa menjalin kolaborasi dengan Berkat Dinasti sebagai objek pengembangan sistem.

Sebagai bentuk kontribusi dalam mendukung transformasi digital UMKM, dikembangkanlah sebuah sistem manajemen database terintegrasi yang dirancang sesuai dengan kebutuhan operasional Berkat Dinasti. Sistem ini mencakup pengelolaan data produk, transaksi penjualan, hingga data pelanggan. Dengan adanya sistem ini, diharapkan proses operasional menjadi lebih efektif, efisien, dan terorganisir dengan baik.

Dokumentasi sistem ini disusun dalam bentuk manual book berjudul “Pengembangan Sistem Manajemen Database Terintegrasi untuk Berkat Dinasti dalam Mendukung Digitalisasi UMKM di Indonesia”. Manual ini difokuskan pada perancangan dan implementasi basis data sebagai inti dari sistem yang dibangun, bukan pada pengembangan aplikasi secara keseluruhan. Hal ini sejalan dengan tujuan utama sebagai bagian dari tugas akhir yang akan diajukan untuk memperoleh Hak Kekayaan Intelektual (HKI).

Basis data yang dikembangkan diberi nama *berkat\_dinasti\_db*, dan terdiri atas 14 tabel utama yang mencakup seluruh kebutuhan operasional Berkat Dinasti, yaitu: data pengguna (admin, kasir, dan pengelola usaha lainnya), kategori, produk, varian\_produk, zona, pelanggan, pesanan, detail\_pesanan, pembayaran, pengiriman, tracking\_log, pengeluaran, transaksi\_kas, serta setting. Dengan struktur yang terintegrasi, sistem ini diharapkan mampu mendukung skalabilitas serta pengembangan sistem di masa mendatang.

Melalui penyusunan manual book ini, diharapkan pihak Berkat Dinasti sebagai pengguna sistem dapat memahami struktur dan alur kerja basis data yang digunakan. Selain itu, dengan adanya sistem ini, diharapkan dapat meningkatkan efektivitas operasional serta menjadi bentuk kontribusi mahasiswa dalam mendukung digitalisasi UMKM di Indonesia melalui solusi teknologi yang praktis, aplikatif, dan dapat dikembangkan lebih lanjut.

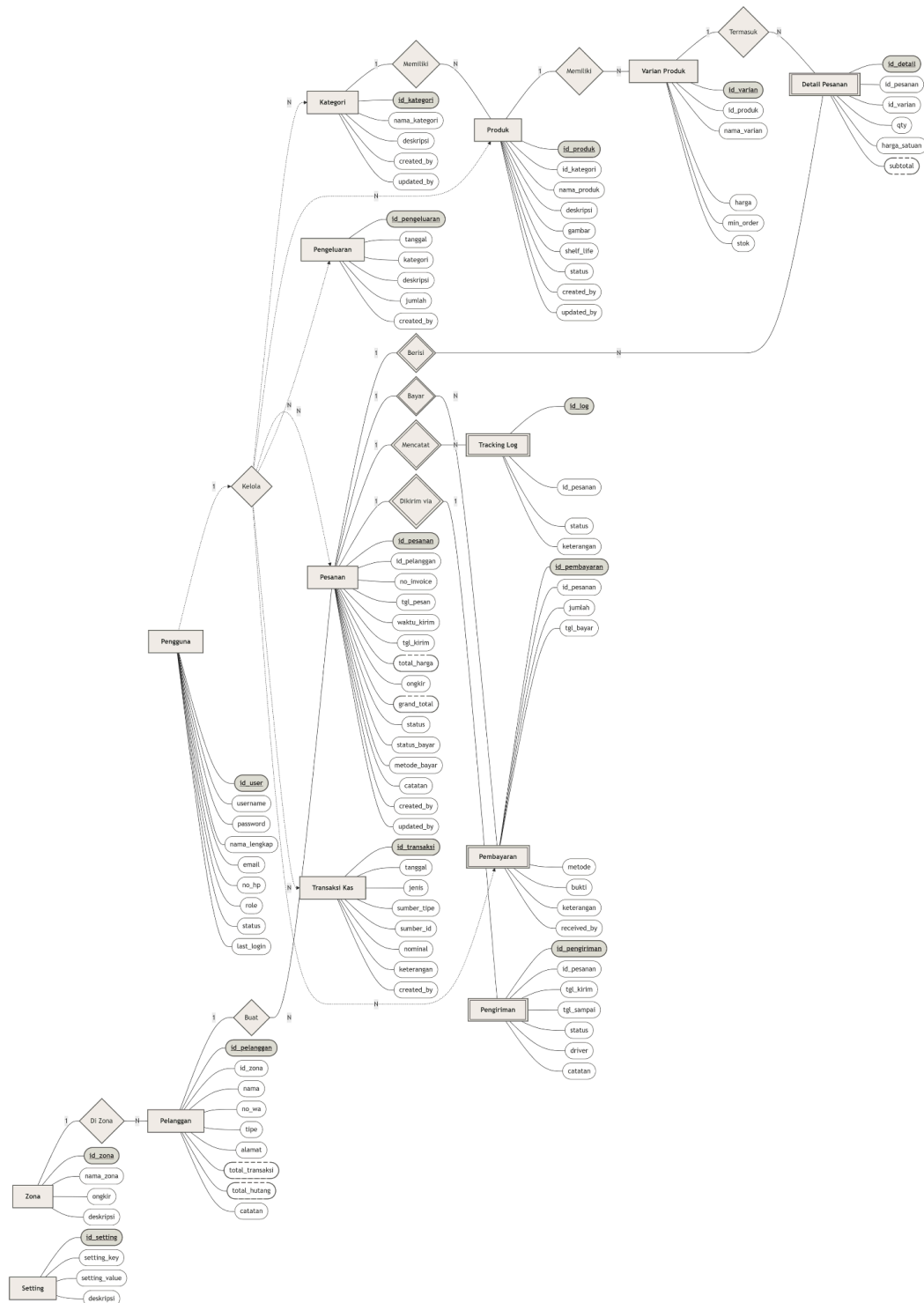
### **III. Tujuan**

Penyusunan *manual book* ini bertujuan untuk:

- a) Mengimplementasikan konsep basis data relasional ke dalam sistem yang dirancang sesuai kebutuhan mitra UMKM Berkat Dinasti.
- b) Menyediakan dokumentasi teknis sebagai panduan dalam memahami dan menggunakan sistem basis data.
- c) Mendukung proses digitalisasi melalui pengelolaan data yang terstruktur guna meningkatkan efektivitas operasional.
- d) Membangun fondasi sistem informasi yang fleksibel sehingga dapat dikembangkan sesuai kebutuhan di masa mendatang.

- e) Memberikan kontribusi dalam mendukung transformasi digital pada sektor UMKM di Indonesia.
- f) Memenuhi persyaratan administrasi dalam pengajuan Hak Kekayaan Intelektual (HKI) atas hasil karya sistem basis data yang dikembangkan.

#### IV. Rancangan Database yang Dikembangkan



Gambar 1.1 Rancangan Basis Data

## V. Source Code

### a. Database berkat\_dinasti\_db

```
CREATE DATABASE IF NOT EXISTS berkat_dinasti_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE berkat_dinasti_db;
```

Bagian ini menetapkan basis data utama aplikasi. Penggunaan `utf8mb4` dan `utf8mb4\_unicode\_ci` memastikan penyimpanan data teks berjalan konsisten dan mendukung karakter lengkap. Perintah `USE` mengarahkan seluruh objek berikutnya ke basis data yang sama.

### b. Tabel pengguna

```
CREATE TABLE pengguna (
  id_user INT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL COMMENT 'Hashed password',
  nama_lengkap VARCHAR(150) NOT NULL,
  email VARCHAR(100),
  no_hp VARCHAR(20),
  role ENUM('admin', 'kasir', 'driver') DEFAULT 'kasir',
  status ENUM('aktif', 'nonaktif') DEFAULT 'aktif',
  last_login DATETIME,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Tabel `pengguna` menyimpan akun yang mengoperasikan sistem. Kolom `username` bersifat unik sebagai identitas login, sedangkan `password` menyimpan hash kata sandi. Kolom `role` membedakan hak akses, dan `status` menunjukkan kondisi akun aktif atau nonaktif.

### c. Tabel kategori

```
CREATE TABLE kategori (
  id_kategori INT PRIMARY KEY AUTO_INCREMENT,
  nama_kategori VARCHAR(100) NOT NULL,
  deskripsi TEXT,
  created_by INT,
  updated_by INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Tabel `kategori` digunakan untuk mengelompokkan produk, misalnya kategori roti hajat dan roti manis. Kolom `created\_by` dan `updated\_by` membantu pencatatan aktivitas pengelolaan data agar tetap dapat ditelusuri.

#### d. Tabel produk

```
CREATE TABLE produk (  
  id_produk INT PRIMARY KEY AUTO_INCREMENT,  
  id_kategori INT NOT NULL,  
  nama_produk VARCHAR(150) NOT NULL,  
  deskripsi TEXT,  
  gambar VARCHAR(255),  
  shelf_life INT DEFAULT 7 COMMENT 'Masa kedaluwarsa dalam hari',  
  status ENUM('tersedia', 'tidak_tersedia') DEFAULT 'tersedia',  
  created_by INT,  
  updated_by INT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  CONSTRAINT fk_produk_kategori  
    FOREIGN KEY (id_kategori) REFERENCES kategori(id_kategori)  
    ON DELETE RESTRICT ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

Tabel `produk` menyimpan data utama barang yang dijual. Setiap produk terhubung dengan satu kategori melalui `id\_kategori`. Kolom `shelf\_life` mencatat masa simpan produk, sedangkan `status` digunakan untuk menandai ketersediaan.

#### e. Tabel varian\_produk

```
CREATE TABLE varian_produk (  
  id_varian INT PRIMARY KEY AUTO_INCREMENT,  
  id_produk INT NOT NULL,  
  nama_varian VARCHAR(100) NOT NULL,  
  harga DECIMAL(10,2) NOT NULL,  
  min_order INT DEFAULT 1,  
  stok INT DEFAULT 0,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  CONSTRAINT fk_varian_produk  
    FOREIGN KEY (id_produk) REFERENCES produk(id_produk)  
    ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

Tabel `varian\_produk` menyimpan variasi produk, seperti bentuk kemasan atau ukuran jual. Satu produk dapat memiliki beberapa varian dengan harga berbeda. Kolom `min\_order` dipakai untuk menerapkan batas pembelian minimal.

#### f. Tabel zona

```
CREATE TABLE zona (  
  id_zona INT PRIMARY KEY AUTO_INCREMENT,  
  nama_zona VARCHAR(100) NOT NULL,  
  deskripsi TEXT,  
  ongkir DECIMAL(10,2) DEFAULT 0.00,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

Tabel `zona` menyimpan wilayah pengiriman beserta ongkos kirim standar. Data ini menjadi dasar perhitungan ongkir pada saat pesanan dibuat.

#### g. Tabel pelanggan

```
CREATE TABLE pelanggan (  
  id_pelanggan INT PRIMARY KEY AUTO_INCREMENT,  
  id_zona INT,  
  nama VARCHAR(150) NOT NULL,  
  alamat TEXT NOT NULL,  
  no_wa VARCHAR(20) NOT NULL,  
  tipe ENUM('retail', 'reseller') DEFAULT 'retail',  
  total_transaksi DECIMAL(15,2) DEFAULT 0.00,  
  total_hutang DECIMAL(15,2) DEFAULT 0.00,  
  catatan TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  CONSTRAINT fk_pelanggan_zona  
    FOREIGN KEY (id_zona) REFERENCES zona(id_zona)  
    ON DELETE SET NULL ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

Tabel `pelanggan` menyimpan data pelanggan yang melakukan pembelian. Kolom `tipe` membedakan pelanggan retail dan reseller. Kolom `total\_transaksi` dan `total\_hutang` digunakan untuk mencatat akumulasi belanja dan piutang pelanggan.

#### h. Tabel pesanan

```
CREATE TABLE pesanan (  
  id_pesanan INT PRIMARY KEY AUTO_INCREMENT,  
  id_pelanggan INT NOT NULL,  
  no_invoice VARCHAR(50) NOT NULL UNIQUE,  
  tgl_pesan DATE NOT NULL,  
  tgl_kirim DATE,  
  waktu_kirim TIME,  
  total_harga DECIMAL(15,2) NOT NULL DEFAULT 0.00,  
  ongkir DECIMAL(10,2) DEFAULT 0.00,  
  grand_total DECIMAL(15,2) NOT NULL DEFAULT 0.00,  
  status ENUM('pending', 'proses', 'kirim', 'selesai', 'batal') DEFAULT 'pending',  
  status_bayar ENUM('belum_bayar', 'dp', 'lunas') DEFAULT 'belum_bayar',  
  metode_bayar ENUM('cash', 'transfer') DEFAULT 'cash',  
  catatan TEXT,  
  created_by INT,  
  updated_by INT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  CONSTRAINT fk_pesanan_pelanggan  
    FOREIGN KEY (id_pelanggan) REFERENCES pelanggan(id_pelanggan)  
    ON DELETE RESTRICT ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

Tabel `pesanan` merupakan header transaksi. Seluruh informasi utama order dicatat di sini, mulai dari pelanggan, jadwal kirim, status proses, status pembayaran, hingga total nilai transaksi.

#### i. Tabel detail\_pesanan

```
CREATE TABLE detail_pesanan (  
  id_detail INT PRIMARY KEY AUTO_INCREMENT,  
  id_pesanan INT NOT NULL,  
  id_varian INT NOT NULL,  
  qty INT NOT NULL,  
  harga_satuan DECIMAL(10,2) NOT NULL,  
  subtotal DECIMAL(15,2) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_detail_pesanan  
    FOREIGN KEY (id_pesanan) REFERENCES pesanan(id_pesanan)  
    ON DELETE CASCADE ON UPDATE CASCADE,  
  CONSTRAINT fk_detail_varian  
    FOREIGN KEY (id_varian) REFERENCES varian_produk(id_varian)  
    ON DELETE RESTRICT ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

Tabel `detail\_pesanan` menyimpan rincian item pada setiap pesanan. Satu pesanan dapat memiliki banyak detail. Kolom `subtotal` mencatat hasil perkalian kuantitas dan harga satuan.

#### j. Tabel pembayaran

```
CREATE TABLE pembayaran (  
  id_pembayaran INT PRIMARY KEY AUTO_INCREMENT,  
  id_pesanan INT NOT NULL,  
  jumlah DECIMAL(15,2) NOT NULL,  
  tgl_bayar DATETIME NOT NULL,  
  metode ENUM('cash', 'transfer') NOT NULL,  
  bukti VARCHAR(255),  
  keterangan TEXT,  
  received_by INT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_pembayaran_pesanan  
    FOREIGN KEY (id_pesanan) REFERENCES pesanan(id_pesanan)  
    ON DELETE RESTRICT ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

Tabel `pembayaran` mendukung transaksi bertahap, baik DP, cicilan, maupun pelunasan. Satu pesanan dapat memiliki lebih dari satu pembayaran. Kolom `received\_by` mengaitkan transaksi dengan pengguna yang menerima pembayaran.

#### k. Tabel pengiriman

```
CREATE TABLE pengiriman (  
  id_pengiriman INT PRIMARY KEY AUTO_INCREMENT,  
  id_pesanan INT NOT NULL UNIQUE,  
  tgl_kirim DATETIME,  
  tgl_sampai DATETIME,  
  status ENUM('menunggu', 'dalam_perjalanan', 'sampai', 'gagal') DEFAULT 'menunggu',  
  driver VARCHAR(100),  
  catatan TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  CONSTRAINT fk_pengiriman_pesanan  
    FOREIGN KEY (id_pesanan) REFERENCES pesanan(id_pesanan)  
    ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

Tabel `pengiriman` menyimpan status distribusi pesanan. Satu pesanan hanya memiliki satu data pengiriman agar status pengiriman tetap konsisten.

### l. Tabel tracking\_log

```
CREATE TABLE tracking_log (  
  id_log INT PRIMARY KEY AUTO_INCREMENT,  
  id_pesanan INT NOT NULL,  
  status VARCHAR(50) NOT NULL,  
  keterangan TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_tracking_pesanan  
    FOREIGN KEY (id_pesanan) REFERENCES pesanan(id_pesanan)  
    ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

Tabel `tracking\_log` berfungsi sebagai riwayat perubahan status pesanan maupun status pembayaran. Data ini mendukung pelacakan proses dan audit transaksi.

### m. Tabel pengeluaran

```
CREATE TABLE pengeluaran (  
  id_pengeluaran INT PRIMARY KEY AUTO_INCREMENT,  
  tanggal DATE NOT NULL,  
  kategori ENUM('bahan_baku', 'operasional', 'gaji', 'transportasi', 'lainnya') NOT NULL,  
  deskripsi TEXT NOT NULL,  
  jumlah DECIMAL(15,2) NOT NULL,  
  created_by INT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_pengeluaran_user  
    FOREIGN KEY (created_by) REFERENCES pengguna(id_user)  
    ON DELETE SET NULL ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

Tabel `pengeluaran` mencatat biaya operasional usaha. Pengelompokan kategori membantu penyusunan laporan keuangan yang lebih terstruktur.

### n. Tabel transaksi\_kas

```
CREATE TABLE transaksi_kas (  
  id_transaksi INT PRIMARY KEY AUTO_INCREMENT,  
  tanggal DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  jenis ENUM('masuk', 'keluar') NOT NULL,  
  sumber_tipe ENUM('pembayaran', 'pengeluaran', 'penyesuaian') NOT NULL,  
  sumber_id INT,  
  nominal DECIMAL(15,2) NOT NULL,  
  keterangan TEXT,  
  dibuat_oleh INT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_transaksi_kas_user  
    FOREIGN KEY (dibuat_oleh) REFERENCES pengguna(id_user)  
    ON DELETE SET NULL ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

Tabel `transaksi\_kas` merupakan buku kas terpusat untuk arus kas masuk dan keluar. Data pada tabel ini digunakan sebagai dasar perhitungan saldo kas.

**o. Tabel setting**

```
CREATE TABLE setting (  
  id_setting INT PRIMARY KEY AUTO_INCREMENT,  
  setting_key VARCHAR(100) NOT NULL UNIQUE,  
  setting_value TEXT,  
  deskripsi TEXT,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

Tabel `setting` menyimpan konfigurasi aplikasi yang bersifat dinamis, seperti identitas toko, nomor kontak, kapasitas angkut, dan jam operasional.

**p. Relasi antar tabel**

Relasi antar tabel disusun menggunakan foreign key untuk menjaga integritas data. Relasi utamanya meliputi:

```
`produk.id_kategori` mengarah ke `kategori.id_kategori`  
`varian_produk.id_produk` mengarah ke `produk.id_produk`  
`pelanggan.id_zona` mengarah ke `zona.id_zona`  
`pesanan.id_pelanggan` mengarah ke `pelanggan.id_pelanggan`  
`detail_pesanan.id_pesanan` mengarah ke `pesanan.id_pesanan`  
`detail_pesanan.id_varian` mengarah ke `varian_produk.id_varian`  
`pembayaran.id_pesanan` mengarah ke `pesanan.id_pesanan`  
`pengiriman.id_pesanan` mengarah ke `pesanan.id_pesanan`  
`tracking_log.id_pesanan` mengarah ke `pesanan.id_pesanan`  
`pengeluaran.created_by` mengarah ke `pengguna.id_user`  
`transaksi_kas.dibuat_oleh` mengarah ke `pengguna.id_user`
```

Selain relasi utama tersebut, kolom audit pada beberapa tabel juga terhubung ke `pengguna` agar aktivitas operasional dapat ditelusuri dengan baik.

**q. Trigger otomatis**

Trigger digunakan untuk menjaga konsistensi data dan mengotomatisasi proses penting dalam sistem Berkat Dinasti. Setiap trigger dirancang untuk memastikan integritas referensial, pembaruan data terkait, dan pencatatan aktivitas otomatis.

1. `trg\_before\_insert\_pesanan`

Trigger ini berjalan sebelum record pesanan dimasukkan untuk membuat nomor invoice otomatis berdasarkan tanggal dan urutan harian.

```

DELIMITER //
CREATE TRIGGER trg_before_insert_pesanan
BEFORE INSERT ON pesanan
FOR EACH ROW
BEGIN
    DECLARE next_num INT;
    DECLARE today_date VARCHAR(8);

    SET today_date = DATE_FORMAT(NEW.tgl_pesanan, '%Y%m%d');

    SELECT COALESCE(MAX(CAST(SUBSTRING(no_invoice, -4) AS UNSIGNED)), 0) + 1
    INTO next_num
    FROM pesanan
    WHERE no_invoice LIKE CONCAT('INV-', today_date, '%');

    SET NEW.no_invoice = CONCAT('INV-', today_date, '-', LPAD(next_num, 4, '0'));
END//
DELIMITER ;

```

## 2. `trg\_before\_insert\_detail`

Trigger ini menghitung `subtotal` secara otomatis saat detail pesanan ditambahkan dengan rumus:  $\text{qty} \times \text{harga\_satuan}$ .

```

DELIMITER //
CREATE TRIGGER trg_before_insert_detail
BEFORE INSERT ON detail_pesanan
FOR EACH ROW
BEGIN
    SET NEW.subtotal = NEW.qty * NEW.harga_satuan;
END//
DELIMITER ;

```

## 3. `trg\_after\_insert\_detail`

Trigger ini memperbarui `total\_harga` dan `grand\_total` pada pesanan setelah item pesanan bertambah.

```

DELIMITER //
CREATE TRIGGER trg_after_insert_detail
AFTER INSERT ON detail_pesanan
FOR EACH ROW
BEGIN
    DECLARE total DECIMAL(15,2);
    DECLARE ongkir_val DECIMAL(10,2);

    SELECT SUM(subtotal) INTO total
    FROM detail_pesanan
    WHERE id_pesanan = NEW.id_pesanan;

    SELECT ongkir INTO ongkir_val
    FROM pesanan
    WHERE id_pesanan = NEW.id_pesanan;

    UPDATE pesanan
    SET total_harga = COALESCE(total, 0),
        grand_total = COALESCE(total, 0) + COALESCE(ongkir_val, 0)
    WHERE id_pesanan = NEW.id_pesanan;
END//
DELIMITER ;

```

#### 4. `trg\_after\_delete\_detail`

Trigger ini menyesuaikan kembali total pesanan ketika detail pesanan dihapus.

```

DELIMITER //
CREATE TRIGGER trg_after_delete_detail
AFTER DELETE ON detail_pesanan
FOR EACH ROW
BEGIN
    DECLARE total DECIMAL(15,2);
    DECLARE ongkir_val DECIMAL(10,2);

    SELECT COALESCE(SUM(subtotal), 0) INTO total
    FROM detail_pesanan
    WHERE id_pesanan = OLD.id_pesanan;

    SELECT ongkir INTO ongkir_val
    FROM pesanan
    WHERE id_pesanan = OLD.id_pesanan;

    UPDATE pesanan
    SET total_harga = total,
        grand_total = total + COALESCE(ongkir_val, 0)
    WHERE id_pesanan = OLD.id_pesanan;
END//
DELIMITER ;

```

5. `trg\_after\_insert\_pembayaran`

Trigger ini mencatat pembayaran ke tabel `transaksi\_kas` sebagai kas masuk dan memperbarui status pembayaran pesanan.

```
DELIMITER //
CREATE TRIGGER trg_after_insert_pembayaran
AFTER INSERT ON pembayaran
FOR EACH ROW
BEGIN
    DECLARE total_dibayar DECIMAL(15,2);
    DECLARE grand DECIMAL(15,2);

    INSERT INTO transaksi_kas (
        tanggal,
        jenis,
        sumber_tipe,
        sumber_id,
        nominal,
        keterangan,
        dibuat_oleh
    ) VALUES (
        NEW.tgl_bayar,
        'masuk',
        'pembayaran',
        NEW.id_pembayaran,
        NEW.jumlah,
        CONCAT('Pembayaran untuk pesanan #', NEW.id_pesanan),
        NEW.received_by
    );

    SELECT SUM(jumlah) INTO total_dibayar
    FROM pembayaran
    WHERE id_pesanan = NEW.id_pesanan;

    SELECT grand_total INTO grand
    FROM pesanan
    WHERE id_pesanan = NEW.id_pesanan;

    IF total_dibayar >= grand THEN
        UPDATE pesanan
        SET status_bayar = 'lunas'
        WHERE id_pesanan = NEW.id_pesanan;
    ELSEIF total_dibayar > 0 THEN
        UPDATE pesanan
        SET status_bayar = 'dp'
        WHERE id_pesanan = NEW.id_pesanan;
    END IF;
END//
DELIMITER ;
```

6. `trg\_after\_update\_pesanan`

Trigger ini mencatat setiap perubahan status pesanan dan status

pembayaran ke tabel `tracking\_log` untuk keperluan audit trail.

```
DELIMITER //
CREATE TRIGGER trg_after_update_pesanan
AFTER UPDATE ON pesanan
FOR EACH ROW
BEGIN
    IF OLD.status != NEW.status THEN
        INSERT INTO tracking_log (id_pesanan, status, keterangan)
        VALUES (
            NEW.id_pesanan,
            NEW.status,
            CONCAT('Status berubah dari ', OLD.status, ' ke ', NEW.status, '')
        );
    END IF;

    IF OLD.status_bayar != NEW.status_bayar THEN
        INSERT INTO tracking_log (id_pesanan, status, keterangan)
        VALUES (
            NEW.id_pesanan,
            CONCAT('bayar_', NEW.status_bayar),
            CONCAT('Status pembayaran berubah dari ', OLD.status_bayar, ' ke ', NEW.status_bayar, '')
        );
    END IF;
END//
DELIMITER ;
```

#### 7. `trg\_after\_pesanan\_selesai`

Trigger ini menambah total transaksi pelanggan dan menghitung piutang jika pesanan belum lunas saat status pesanan menjadi "selesai".

```
DELIMITER //
CREATE TRIGGER trg_after_pesanan_selesai
AFTER UPDATE ON pesanan
FOR EACH ROW
BEGIN
    DECLARE sisa_hutang DECIMAL(15,2);
    DECLARE total_bayar DECIMAL(15,2);

    IF NEW.status = 'selesai' AND OLD.status != 'selesai' THEN
        UPDATE pelanggan
        SET total_transaksi = total_transaksi + NEW.grand_total
        WHERE id_pelanggan = NEW.id_pelanggan;
    END IF;

    IF NEW.status = 'selesai' AND OLD.status != 'selesai' AND NEW.status_bayar != 'lunas' THEN
        SELECT COALESCE(SUM(jumlah), 0) INTO total_bayar
        FROM pembayaran WHERE id_pesanan = NEW.id_pesanan;

        SET sisa_hutang = GREATEST(NEW.grand_total - total_bayar, 0);

        UPDATE pelanggan
        SET total_hutang = total_hutang + sisa_hutang
        WHERE id_pelanggan = NEW.id_pelanggan;
    END IF;
END//
DELIMITER ;
```

#### 8. `trg\_after\_insert\_pesanan`

Trigger ini membuat data awal pengiriman dan log status awal saat pesanan baru dibuat.

```

DELIMITER //
CREATE TRIGGER trg_after_insert_pesanan
AFTER INSERT ON pesanan
FOR EACH ROW
BEGIN
    INSERT INTO pengiriman (id_pesanan, status)
    VALUES (NEW.id_pesanan, 'menunggu');

    INSERT INTO tracking_log (id_pesanan, status, keterangan)
    VALUES (NEW.id_pesanan, 'pending', 'Pesanan baru dibuat');
END//
DELIMITER ;

```

9. `trg\_after\_insert\_pengeluaran`

Trigger ini mencatat pengeluaran operasional ke tabel `transaksi\_kas` sebagai kas keluar.

```

DELIMITER //
CREATE TRIGGER trg_after_insert_pengeluaran
AFTER INSERT ON pengeluaran
FOR EACH ROW
BEGIN
    INSERT INTO transaksi_kas (
        tanggal,
        jenis,
        sumber_tipe,
        sumber_id,
        nominal,
        keterangan,
        dibuat_oleh
    ) VALUES (
        CONCAT(NEW.tanggal, ' 00:00:00'),
        'keluar',
        'pengeluaran',
        NEW.id_pengeluaran,
        NEW.jumlah,
        NEW.deskripsi,
        NEW.created_by
    );
END//
DELIMITER ;

```

**r. View laporan**

View disediakan untuk memudahkan penyajian data operasional dan keuangan tanpa harus menulis query kompleks berulang. View-view berikut dirancang untuk mendukung keputusan bisnis dan pelaporan:

1. `v\_pesanan\_lengkap`

View ini menampilkan ringkasan pesanan yang digabungkan dengan

data pelanggan, zona pengiriman, dan status pengiriman. Memudahkan visualisasi pesanan secara menyeluruh.

```
CREATE VIEW v_pesanan_lengkap AS
SELECT
    p.id_pesanan,
    p.no_invoice,
    p.tgl_pesan,
    p.tgl_kirim,
    p.waktu_kirim,
    p.total_harga,
    p.ongkir,
    p.grand_total,
    p.status,
    p.status_bayar,
    p.metode_bayar,
    p.catatan,
    pel.id_pelanggan,
    pel.nama AS nama_pelanggan,
    pel.alamat,
    pel.no_wa,
    pel.tipe AS tipe_pelanggan,
    z.nama_zona,
    pg.status AS status_pengiriman,
    pg.driver
FROM pesanan p
JOIN pelanggan pel ON p.id_pelanggan = pel.id_pelanggan
LEFT JOIN zona z ON pel.id_zona = z.id_zona
LEFT JOIN pengiriman pg ON p.id_pesanan = pg.id_pesanan;
```

## 2. `v\_detail\_pesanan\_produk`

View ini menampilkan rincian item pesanan beserta informasi produk, varian, kuantitas, harga satuan, dan subtotal untuk analisis detail penjualan.

```
CREATE VIEW v_detail_pesanan_produk AS
SELECT
    dp.id_detail,
    dp.id_pesanan,
    p.no_invoice,
    pr.nama_produk,
    vp.nama_varian,
    dp.qty,
    dp.harga_satuan,
    dp.subtotal
FROM detail_pesanan dp
JOIN pesanan p ON dp.id_pesanan = p.id_pesanan
JOIN varian_produk vp ON dp.id_varian = vp.id_varian
JOIN produk pr ON vp.id_produk = pr.id_produk;
```

### 3. `v\_laporan\_harian`

View ini menghasilkan ringkasan penjualan harian dengan metrik jumlah order, nilai penjualan, total yang lunas, dan total piutang untuk memantau performa harian.

```
CREATE VIEW v_laporan_harian AS
SELECT
    DATE(tgl_pesan) AS tanggal,
    COUNT(id_pesanan) AS total_order,
    SUM(CASE WHEN status = 'selesai' THEN grand_total ELSE 0 END) AS total_penjualan,
    SUM(CASE WHEN status_bayar = 'lunas' THEN grand_total ELSE 0 END) AS total_lunas,
    SUM(CASE WHEN status_bayar != 'lunas' THEN grand_total ELSE 0 END) AS total_piutang
FROM pesanan
WHERE status != 'batal'
GROUP BY DATE(tgl_pesan)
ORDER BY tanggal DESC;
```

### 4. `v\_piutang`

View ini menampilkan daftar pelanggan dengan detail sisa hutang, membantu manajemen kredit dan koleksi.

```
CREATE VIEW v_piutang AS
SELECT
    pel.id_pelanggan,
    pel.nama,
    pel.no_wa,
    pel.total_hutang,
    COUNT(p.id_pesanan) AS jumlah_pesanan_belum_lunas,
    SUM(p.grand_total) AS total_pesanan,
    SUM(COALESCE(pb.total_bayar, 0)) AS total_terbayar,
    SUM(p.grand_total) - SUM(COALESCE(pb.total_bayar, 0)) AS sisa_hutang
FROM pelanggan pel
JOIN pesanan p ON pel.id_pelanggan = p.id_pelanggan
LEFT JOIN (
    SELECT id_pesanan, SUM(jumlah) AS total_bayar
    FROM pembayaran
    GROUP BY id_pesanan
) pb ON p.id_pesanan = pb.id_pesanan
WHERE p.status_bayar != 'lunas' AND p.status != 'batal'
GROUP BY pel.id_pelanggan, pel.nama, pel.no_wa, pel.total_hutang
HAVING sisa_hutang > 0;
```

### 5. `v\_katalog`

View ini menyajikan daftar produk aktif beserta kategori, varian, harga, dan stok untuk keperluan penampilan katalog produk.

```
CREATE VIEW v_katalog AS
SELECT
    p.id_produk,
    k.nama_kategori,
    p.nama_produk,
    p.deskripsi,
    p.gambar,
    p.shelf_life,
    p.status,
    v.id_varian,
    v.nama_varian,
    v.harga,
    v.min_order,
    v.stok
FROM produk p
JOIN kategori k ON p.id_kategori = k.id_kategori
LEFT JOIN varian_produk v ON p.id_produk = v.id_produk
WHERE p.status = 'tersedia'
ORDER BY k.nama_kategori, p.nama_produk, v.harga;
```

#### 6. `v\_ringkasan\_kas`

View ini merangkum total kas masuk, kas keluar, dan saldo akhir untuk memberikan gambaran cepat posisi kas perusahaan.

```
CREATE VIEW v_ringkasan_kas AS
SELECT
    COALESCE(SUM(CASE WHEN jenis = 'masuk' THEN nominal ELSE 0 END), 0) AS total_masuk,
    COALESCE(SUM(CASE WHEN jenis = 'keluar' THEN nominal ELSE 0 END), 0) AS total_keluar,
    COALESCE(SUM(CASE WHEN jenis = 'masuk' THEN nominal ELSE -nominal END), 0) AS saldo_akhir
FROM transaksi_kas;
```

#### 7. `v\_laporan\_kas\_harian`

View ini menampilkan ringkasan pergerakan kas per tanggal dengan detail pemasukan, pengeluaran, dan saldo harian untuk analisis cash flow.

```
CREATE VIEW v_laporan_kas_harian AS
SELECT
    DATE(tanggal) AS tanggal,
    SUM(CASE WHEN jenis = 'masuk' THEN nominal ELSE 0 END) AS pemasukan,
    SUM(CASE WHEN jenis = 'keluar' THEN nominal ELSE 0 END) AS pengeluaran,
    SUM(CASE WHEN jenis = 'masuk' THEN nominal ELSE -nominal END) AS saldo_harian
FROM transaksi_kas
GROUP BY DATE(tanggal)
ORDER BY tanggal DESC;
```

### s. Storage procedure

Stored procedure digunakan untuk mengotomatisasi proses bisnis inti agar

berjalan melalui alur yang baku dan tervalidasi. Setiap prosedur dirancang dengan parameter input/output yang jelas dan logika bisnis yang terintegrasi.

1. ``sp_create_pesanan``

Procedure ini membuat pesanan baru dan secara otomatis menghitung biaya ongkir berdasarkan zona pelanggan. Nomor invoice dibuat melalui trigger.

```
DELIMITER //
CREATE PROCEDURE sp_create_pesanan(
    IN p_id_pelanggan INT,
    IN p_tgl_kirim DATE,
    IN p_waktu_kirim TIME,
    IN p_metode_bayar VARCHAR(20),
    IN p_catatan TEXT,
    OUT p_id_pesanan INT
)
BEGIN
    DECLARE v_ongkir DECIMAL(10,2);

    -- Get ongkir based on pelanggan zona
    SELECT COALESCE(z.ongkir, 0) INTO v_ongkir
    FROM pelanggan pel
    LEFT JOIN zona z ON pel.id_zona = z.id_zona
    WHERE pel.id_pelanggan = p_id_pelanggan;

    INSERT INTO pesanan (
        id_pelanggan,
        tgl_pesan,
        tgl_kirim,
        waktu_kirim,
        ongkir,
        metode_bayar,
        catatan
    ) VALUES (
        p_id_pelanggan,
        CURDATE(),
        p_tgl_kirim,
        p_waktu_kirim,
        v_ongkir,
        p_metode_bayar,
        p_catatan
    );

    SET p_id_pesanan = LAST_INSERT_ID();
END//
DELIMITER ;
```

Parameter:

- Input: ``p_id_pelanggan``, ``p_tgl_kirim``, ``p_waktu_kirim``, ``p_metode_bayar``, ``p_catatan``
- Output: ``p_id_pesanan`` (ID pesanan yang baru dibuat)

## 2. `sp\_add\_detail\_pesanan`

Procedure ini menambahkan detail item pesanan dengan validasi minimum order berdasarkan varian produk.

```
DELIMITER //
CREATE PROCEDURE sp_add_detail_pesanan(
    IN p_id_pesanan INT,
    IN p_id_varian INT,
    IN p_qty INT
)
BEGIN
    DECLARE v_harga DECIMAL(10,2);
    DECLARE v_min_order INT;

    -- Get price and min_order from varian
    SELECT harga, min_order INTO v_harga, v_min_order
    FROM varian_produk
    WHERE id_varian = p_id_varian;

    -- Check minimum order
    IF p_qty < v_min_order THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Quantity kurang dari minimum order';
    END IF;

    INSERT INTO detail_pesanan (id_pesanan, id_varian, qty, harga_satuan, subtotal)
    VALUES (p_id_pesanan, p_id_varian, p_qty, v_harga, p_qty * v_harga);
END//
DELIMITER ;
```

Parameter:

- Input: `p\_id\_pesanan`, `p\_id\_varian`, `p\_qty` (jumlah barang yang dipesan)

## 3. `sp\_update\_status`

Procedure ini memperbarui status pesanan dan secara otomatis menyesuaikan status pengiriman serta timestamp-nya.

```
DELIMITER //
CREATE PROCEDURE sp_update_status(
    IN p_id_pesanan INT,
    IN p_status VARCHAR(20)
)
BEGIN
    UPDATE pesanan
    SET status = p_status
    WHERE id_pesanan = p_id_pesanan;

    -- Also update pengiriman status if needed
    IF p_status = 'kirim' THEN
        UPDATE pengiriman
        SET status = 'dalam_perjalanan', tgl_kirim = NOW()
        WHERE id_pesanan = p_id_pesanan;
    ELSEIF p_status = 'selesai' THEN
        UPDATE pengiriman
        SET status = 'sampai', tgl_sampai = NOW()
        WHERE id_pesanan = p_id_pesanan;
    END IF;
END//
DELIMITER ;
```

Parameter:

- Input: `p\_id\_pesanan`, `p\_status` (status baru: 'pending', 'kirim', 'selesai', 'batal')

#### 4. `sp\_bayar`

Procedure ini mencatat transaksi pembayaran pesanan. Pembaruan status pembayaran (dp/lunas) ditangani secara otomatis oleh trigger `trg\_after\_insert\_pembayaran`.

```
DELIMITER //
CREATE PROCEDURE sp_bayar(
    IN p_id_pesanan INT,
    IN p_jumlah DECIMAL(15,2),
    IN p_metode VARCHAR(20),
    IN p_bukti VARCHAR(255),
    IN p_keterangan TEXT,
    IN p_received_by INT
)
BEGIN
    INSERT INTO pembayaran (
        id_pesanan,
        jumlah,
        tgl_bayar,
        metode,
        bukti,
        keterangan,
        received_by
    ) VALUES (
        p_id_pesanan,
        p_jumlah,
        NOW(),
        p_metode,
        p_bukti,
        p_keterangan,
        p_received_by
    );
END//
DELIMITER ;
```

Parameter:

- Input: `p\_id\_pesanan`, `p\_jumlah`, `p\_metode` (transfer/cash/dll), `p\_bukti`, `p\_keterangan`, `p\_received\_by` (user ID kasir)

#### 5. `sp\_laporan\_penjualan`

Procedure ini menghasilkan laporan penjualan berdasarkan rentang tanggal, menampilkan ringkasan order, nilai penjualan, dan tingkat

penyelesaian.

```
DELIMITER //
CREATE PROCEDURE sp_laporan_penjualan(
    IN p_start_date DATE,
    IN p_end_date DATE
)
BEGIN
    SELECT
        DATE(tgl_pesan) AS tanggal,
        COUNT(*) AS jumlah_order,
        SUM(CASE WHEN status = 'selesai' THEN 1 ELSE 0 END) AS order_selesai,
        SUM(CASE WHEN status = 'batal' THEN 1 ELSE 0 END) AS order_batal,
        SUM(CASE WHEN status = 'selesai' THEN grand_total ELSE 0 END) AS total_penjualan,
        SUM(CASE WHEN status_bayar = 'lunas' AND status = 'selesai' THEN grand_total ELSE 0 END) AS total_lunas
    FROM pesanan
    WHERE tgl_pesan BETWEEN p_start_date AND p_end_date
    GROUP BY DATE(tgl_pesan)
    ORDER BY tanggal;
END//
DELIMITER ;
```

Parameter:

- Input: `p\_start\_date`, `p\_end\_date` (periode laporan yang diinginkan)

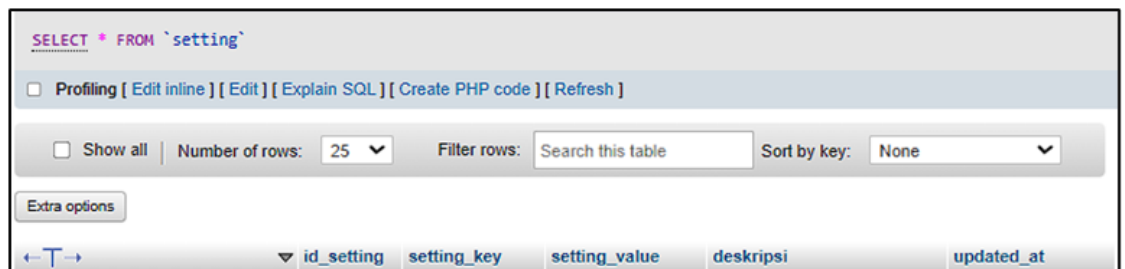
## VI. Hasil Query

### a. Tabel Pengguna



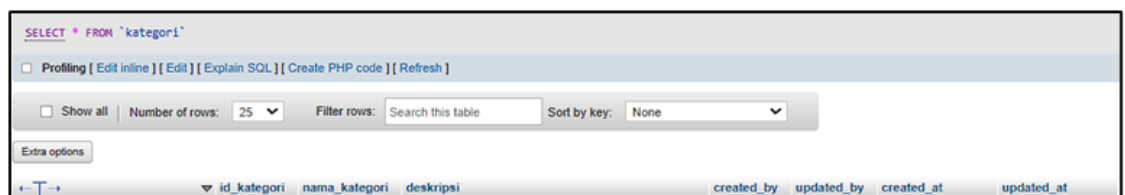
| id_user | username | password                                                 | nama_lengkap  | email | no_hp | role  | status | last_login | created_at          | updated_at          |
|---------|----------|----------------------------------------------------------|---------------|-------|-------|-------|--------|------------|---------------------|---------------------|
| 1       | admin    | \$2y\$10\$8K1p5s1VvqHqYL1YQs7xK0zR8h7L8P3e4Q5w6E7d8T9... | Administrator | NULL  | NULL  | admin | aktif  | NULL       | 2026-04-18 12:52:33 | 2026-04-18 12:52:33 |

### b. Tabel Setting



| id_setting | setting_key | setting_value | deskripsi | updated_at |
|------------|-------------|---------------|-----------|------------|
|------------|-------------|---------------|-----------|------------|

### c. Tabel Kategori



| id_kategori | nama_kategori | deskripsi | created_by | updated_by | created_at | updated_at |
|-------------|---------------|-----------|------------|------------|------------|------------|
|-------------|---------------|-----------|------------|------------|------------|------------|

#### d. Tabel Produk

SELECT \* FROM "produk"

Profiling

Edit inline

Edit

Explain SQL

Create PHP code

Refresh

Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Extra options

|  | id_produk | id_kategori | nama_produk | deskripsi | gambar | shelf_life | status | created_by | updated_by | created_at | updated_at |
|--|-----------|-------------|-------------|-----------|--------|------------|--------|------------|------------|------------|------------|
|--|-----------|-------------|-------------|-----------|--------|------------|--------|------------|------------|------------|------------|

**e. Tabel Varian\_produk**

SELECT \* FROM `varian\_produk`

Profiling

Edit inline

Edit

Explain SQL

Create PHP code

Refresh

Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Extra options

id\_varian

id\_produk

nama\_varian  
Contoh: Kardus, Mika, Satuan

harga

min\_order  
Minimal pembelian

stok

created\_at

updated\_at

**f. Tabel Zona**

SELECT \* FROM `zona`

☐ Profiling [\[ Edit inline \]](#) [\[ Edit \]](#) [\[ Explain SQL \]](#) [\[ Create PHP code \]](#) [\[ Refresh \]](#)

☐ Show all

Number of rows: 

25

Filter rows: 

Search this table

Sort by key: 

None

Extra options

← T →

▼ id\_zona

nama\_zona

deskripsi

ongkir

created\_at

updated\_at

### g. Tabel Pelanggan

```
SELECT * FROM `pelanggan`
```

☐ Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

| id_pelanggan | id_zona | nama | alamat                  | no_wa       | tipe      | total_transaksi<br><small>Akumulasi total belanja</small> | total_hutang<br><small>Total piutang belum lunas</small> | catatan | created_at          | updated_at          |
|--------------|---------|------|-------------------------|-------------|-----------|-----------------------------------------------------------|----------------------------------------------------------|---------|---------------------|---------------------|
| 1            | 1       | Budi | Jl. Merdeka No. 10      | 08123456789 | Pengantar | 1000000                                                   | 0                                                        |         | 2023-10-27 10:00:00 | 2023-10-27 10:00:00 |
| 2            | 1       | Siti | Jl. Sudirman No. 25     | 08987654321 | Penerima  | 0                                                         | 500000                                                   |         | 2023-10-27 10:00:00 | 2023-10-27 10:00:00 |
| 3            | 2       | Andi | Jl. Gatot Subroto No. 5 | 08555555555 | Pengantar | 2000000                                                   | 0                                                        |         | 2023-10-27 10:00:00 | 2023-10-27 10:00:00 |
| 4            | 2       | Rani | Jl. Diponegoro No. 15   | 08777777777 | Penerima  | 0                                                         | 1000000                                                  |         | 2023-10-27 10:00:00 | 2023-10-27 10:00:00 |

#### h. Tabel Pesanan

**SELECT \* FROM `pesanan`**

☐ Profiling [\[ Edit inline \]](#) [\[ Edit \]](#) [\[ Explain SQL \]](#) [\[ Create PHP code \]](#) [\[ Refresh \]](#)

| id_pesanan | id_pelanggan | no_invoice | tgl_pesanan | tgl_kirim | waktu_kirim | total_harga | ongkir | grand_total | status | status_bayar | metode_bayar | catatan | created_by | updated_by | created_at | updated_at |
|------------|--------------|------------|-------------|-----------|-------------|-------------|--------|-------------|--------|--------------|--------------|---------|------------|------------|------------|------------|
|------------|--------------|------------|-------------|-----------|-------------|-------------|--------|-------------|--------|--------------|--------------|---------|------------|------------|------------|------------|

i. Tabel Detail\_pesanan

|                                                                                                             |            |           |     |              |          |            |
|-------------------------------------------------------------------------------------------------------------|------------|-----------|-----|--------------|----------|------------|
| <pre>SELECT * FROM `detail_pesanan`</pre>                                                                   |            |           |     |              |          |            |
| <input type="checkbox"/> Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ] |            |           |     |              |          |            |
| id_detail                                                                                                   | id_pesanan | id_varian | qty | harga_satuan | subtotal | created_at |

j. Tabel Pembayaran

|                                                                                                             |            |        |           |        |                          |            |             |            |  |
|-------------------------------------------------------------------------------------------------------------|------------|--------|-----------|--------|--------------------------|------------|-------------|------------|--|
| <pre>SELECT * FROM `pembayaran`</pre>                                                                       |            |        |           |        |                          |            |             |            |  |
| <input type="checkbox"/> Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ] |            |        |           |        |                          |            |             |            |  |
| id_pembayaran                                                                                               | id_pesanan | jumlah | tgl_bayar | metode | bukti                    | keterangan | received_by | created_at |  |
|                                                                                                             |            |        |           |        | Path file bukti transfer |            |             |            |  |

k. Tabel Pengiriman

|                                                                                                             |            |           |            |        |        |         |            |            |  |
|-------------------------------------------------------------------------------------------------------------|------------|-----------|------------|--------|--------|---------|------------|------------|--|
| <pre>SELECT * FROM `pengiriman`</pre>                                                                       |            |           |            |        |        |         |            |            |  |
| <input type="checkbox"/> Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ] |            |           |            |        |        |         |            |            |  |
| id_pengiriman                                                                                               | id_pesanan | tgl_kirim | tgl_sampai | status | driver | catatan | created_at | updated_at |  |

l. Tabel Tracking\_log

|                                                                                                             |            |        |            |            |
|-------------------------------------------------------------------------------------------------------------|------------|--------|------------|------------|
| <pre>SELECT * FROM `tracking_log`</pre>                                                                     |            |        |            |            |
| <input type="checkbox"/> Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ] |            |        |            |            |
| id_log                                                                                                      | id_pesanan | status | keterangan | created_at |

m. Tabel Pengeluaran

|                                                                                                             |         |          |           |        |            |            |
|-------------------------------------------------------------------------------------------------------------|---------|----------|-----------|--------|------------|------------|
| <pre>SELECT * FROM `pengeluaran`</pre>                                                                      |         |          |           |        |            |            |
| <input type="checkbox"/> Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ] |         |          |           |        |            |            |
| id_pengeluaran                                                                                              | tanggal | kategori | deskripsi | jumlah | created_by | created_at |

n. Tabel Transaksi\_kas

```
SELECT * FROM `transaksi_kas`
```

☐ Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

| id_transaksi | tanggal | jenis | sumber_tipe | sumber_id | nominal | keterangan | created_by | created_at |
|--------------|---------|-------|-------------|-----------|---------|------------|------------|------------|
|--------------|---------|-------|-------------|-----------|---------|------------|------------|------------|

o. Stored Procedure sp\_create\_pesanan call query

```
Run SQL query/queries on database berkat_dinasti_db: ⓘ

1 USE berkat_dinasti_db;
2
3 -- Input parameter
4 SET @p_id_pelanggan = 1;
5 SET @p_tgl_kirim = '2026-04-25';
6 SET @p_waktu_kirim = '09:00:00';
7 SET @p_metode_bayar = 'cash';
8 SET @p_catatan = 'Pesanan untuk acara';
9
10 -- Output parameter
11 SET @p_id_pesanan = 0;
12
13 CALL sp_create_pesanan(
14     @p_id_pelanggan,
15     @p_tgl_kirim,
16     @p_waktu_kirim,
17     @p_metode_bayar,
18     @p_catatan,
19     @p_id_pesanan
20 );
21
22 -- Tampilkan id pesanan yang baru dibuat (penting untuk prosedur berikutnya)
23 SELECT @p_id_pesanan AS id_pesanan_baru;
```

p. Stored Procedure sp\_add\_detail\_pesanan call query

```
Run SQL query/queries on database berkat_dinasti_db: ⓘ

1 USE berkat_dinasti_db;
2
3 -- Gunakan id pesanan dari hasil sp_create_pesanan
4 SET @p_id_pesanan = 1;
5 SET @p_id_varian = 1;
6 SET @p_qty = 10;
7
8 CALL sp_add_detail_pesanan(
9     @p_id_pesanan,
10    @p_id_varian,
11    @p_qty
12 );
13
14 -- Cek hasil detail yang masuk
15 SELECT *
16 FROM detail_pesanan
17 WHERE id_pesanan = @p_id_pesanan;
```

q. Stored Procedure `sp_update_status` call query

```
Run SQL query/queries on database berkat_dinasti_db:

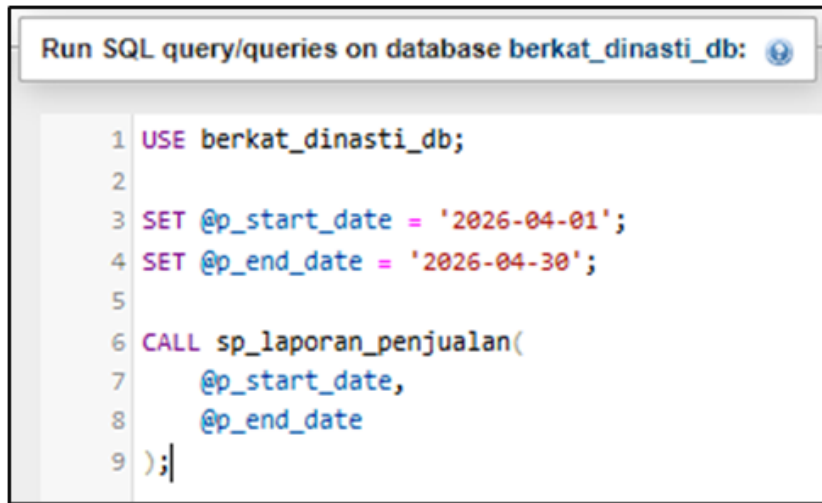
1 USE berkat_dinasti_db;
2
3 SET @p_id_pesanan = 1;
4 SET @p_status = 'kirim'; -- opsi: pending, proses, kirim, selesai, batal
5
6 CALL sp_update_status(
7     @p_id_pesanan,
8     @p_status
9 );
10
11 -- Verifikasi status pesanan dan pengiriman
12 SELECT id_pesanan, status, status_bayar
13 FROM pesanan
14 WHERE id_pesanan = @p_id_pesanan;
15
16 SELECT id_pesanan, status, tgl_kirim, tgl_sampai
17 FROM pengiriman
18 WHERE id_pesanan = @p_id_pesanan;
```

r. Stored Procedure `sp_bayar` call query

```
Run SQL query/queries on database berkat_dinasti_db:

1 USE berkat_dinasti_db;
2
3 SET @p_id_pesanan = 1;
4 SET @p_jumlah = 50000.00;
5 SET @p_metode = 'cash'; -- opsi: cash, transfer
6 SET @p_bukti = NULL; -- isi path jika transfer
7 SET @p_keterangan = 'Pembayaran tahap 1';
8 SET @p_received_by = 1; -- id_user penerima pembayaran
9
10 CALL sp_bayar(
11     @p_id_pesanan,
12     @p_jumlah,
13     @p_metode,
14     @p_bukti,
15     @p_keterangan,
16     @p_received_by
17 );
18
19 -- Verifikasi pembayaran dan update status bayar
20 SELECT *
21 FROM pembayaran
22 WHERE id_pesanan = @p_id_pesanan
23 ORDER BY id_pembayaran DESC;
24
25 SELECT id_pesanan, grand_total, status_bayar
26 FROM pesanan
27 WHERE id_pesanan = @p_id_pesanan;
```

s. Stored Procedure `sp_laporan_penjualan` call query



```
1 USE berkat_dinasti_db;
2
3 SET @p_start_date = '2026-04-01';
4 SET @p_end_date = '2026-04-30';
5
6 CALL sp_laporan_penjualan(
7     @p_start_date,
8     @p_end_date
9 );|
```

## VII. Kesimpulan

Pengembangan Sistem Manajemen Database Terintegrasi ini menjadi pengalaman penting bagi mahasiswa dalam menerapkan konsep basis data relasional sesuai kebutuhan mitra. Bersama mitra Berkat Dinasti, mahasiswa mengimplementasikan normalisasi data, perancangan relasi antar tabel, serta penggunaan trigger dan stored procedure untuk otomatisasi proses bisnis. Proyek ini juga melatih kemampuan analisis terhadap kebutuhan operasional UMKM yang dinamis. Berdasarkan proses tersebut, dapat dipahami bahwa struktur database yang kuat adalah fondasi sistem informasi yang transparan, akurat, dan efisien.

Secara lebih luas, proyek ini menunjukkan peran strategis akademisi dalam mempercepat transformasi digital UMKM di Indonesia. Implementasi database terintegrasi untuk pengelolaan produk, transaksi, pelanggan, dan arus kas menjadi solusi nyata untuk menggantikan pencatatan manual yang rawan kesalahan. Digitalisasi ini mendukung pengambilan keputusan berbasis data dan meningkatkan daya saing usaha di era ekonomi digital. Penyusunan manual book untuk tugas akhir dan pengajuan HKI diharapkan menjadi standar dokumentasi teknis yang aplikatif. Keberhasilan proyek ini menegaskan bahwa sinergi dunia pendidikan dan industri dapat menghasilkan solusi teknologi yang tepat guna, berkelanjutan, dan berdampak nyata bagi penguatan ekonomi digital nasional.